Assignment 7: Kubernetes Deployment of Full-Stack Application

Submitted by: Umair Khan

Course: DevOps — Tutedude

Date: 12 October 2026

GitHub Repository:

https://github.com/Umair-bey/tutedude-assignments/tree/main/Assignment-7

---

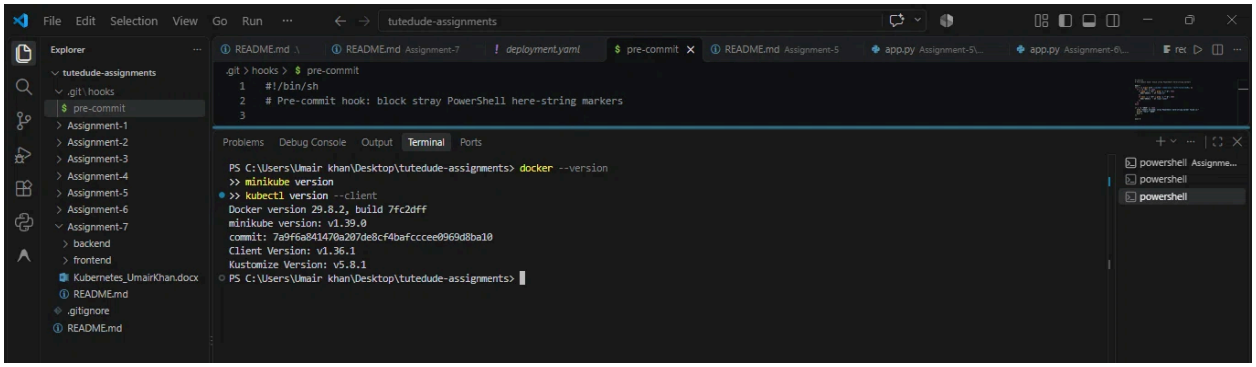
1. Objective

The objective of this assignment is to deploy a full-stack application consisting of a **Flask frontend** and an **Express backend** on a local **Kubernetes cluster** using **Minikube**. This involves containerizing both applications using Docker, writing Kubernetes manifests (Deployments and Services) for each, applying them to the cluster, and verifying that the pods and services are running correctly.

2. Prerequisites

The following tools were installed and verified on the local machine (Windows 11):

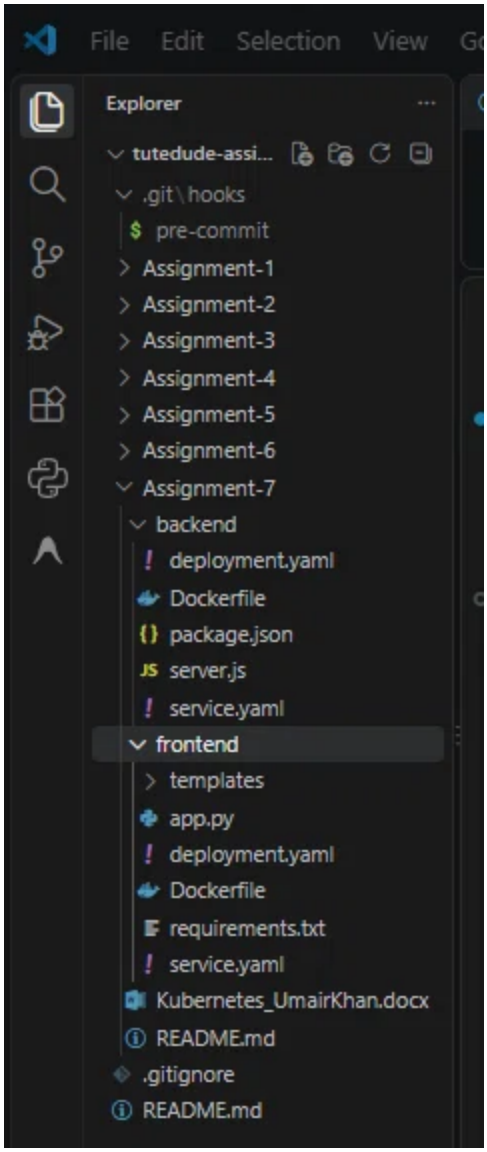
Tool	Version	Purpose
Docker Desktop	v29.8.2	Container runtime for building and running images
Minikube	v1.39.0	Local single-node Kubernetes cluster
kubectl	v1.32.2	Kubernetes command-line tool
Git	Latest	Version control
VS Code	Latest	Code editor and terminal



3. Project Structure

The project source code is organized as follows:

```
Assignment-7/
├── backend/
│   ├── Dockerfile
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── package.json
│   └── server.js
├── frontend/
│   ├── Dockerfile
│   ├── deployment.yaml
│   ├── service.yaml
│   ├── app.py
│   ├── requirements.txt
│   └── templates/
│       └── index.html
├── screenshots/
├── Kubernetes_UmairKhan.docx
└── README.md
```



---

4. Application Architecture

The application consists of two microservices:

- **Frontend (Flask):** Runs on port 5000. Serves an HTML page and makes an HTTP request to the backend service to fetch data.
- **Backend (Express):** Runs on port 3000. Exposes a REST API endpoint `/api` returning a JSON message.

Communication flow:

Browser → frontend-service (NodePort: 5000) → Flask Pod

Flask Pod → backend-service (ClusterIP: 3000) → Express Pod

Kubernetes internal DNS resolves `backend-service` to the correct pod IP, enabling service discovery without hardcoded IPs.

---

5. Step-by-Step Deployment

Step 1: Start Minikube Cluster

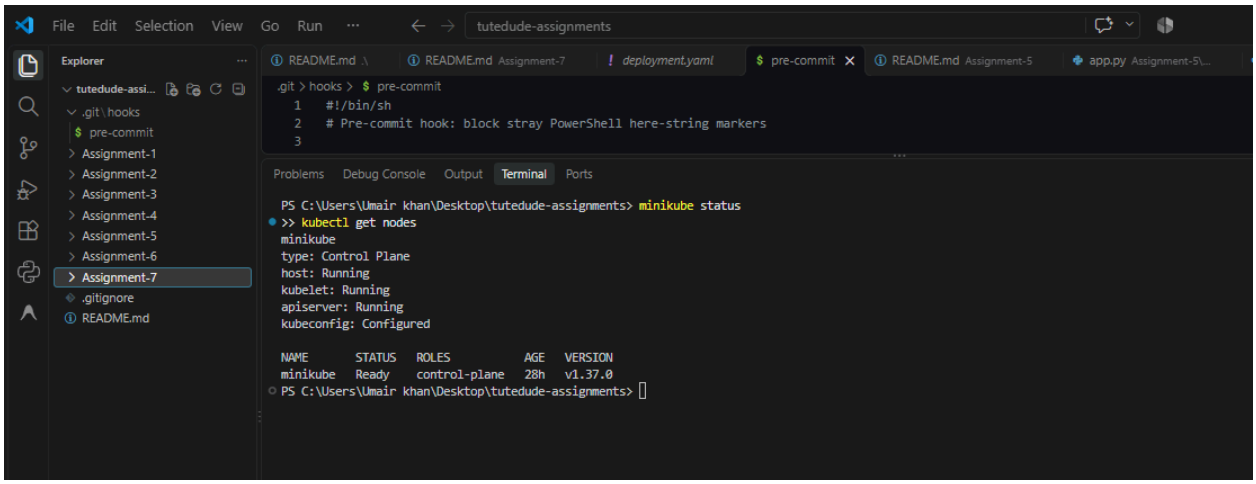
Started the local Kubernetes cluster using the Docker driver:

powershell
minikube start --driver=docker

[INSERT SCREENSHOT: `minikube start` output showing "Done!"]

Verify cluster status:

powershell
minikube status
kubectl get nodes



The node `minikube` shows status **Ready** with role **control-plane**.

Step 2: Containerize the Backend (Express)

`backend/Dockerfile`:

dockerfile
```
FROM node:18-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

`backend/server.js` (excerpt):

javascript
```
const express = require('express');
const cors = require('cors');

const app = express();
app.use(cors());
app.use(express.json());

app.get('/api', (req, res) => {
  res.json({ message: "Hello from Express Backend on Kubernetes!", status: "success" });
});

app.get('/api/health', (req, res) => {
  res.json({ status: "healthy" });
});

const PORT = process.env.PORT || 3000;
app.listen(PORT, () => console.log(`Backend running on port ${PORT}`));
```

Step 3: Containerize the Frontend (Flask)

`frontend/Dockerfile`:

dockerfile
```
FROM python:3.10-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
```

```
COPY . .
EXPOSE 5000
CMD ["python", "app.py"]
```

`frontend/app.py` (excerpt):

```
python
from flask import Flask, render_template
import requests
import os

app = Flask(__name__)
BACKEND_URL = os.getenv("BACKEND_URL", "http://backend-service:3000")

@app.route('/')
def home():
    try:
        r = requests.get(f"{BACKEND_URL}/api", timeout=5)
        data = r.json()
    except Exception as e:
        data = {"message": f"Backend not reachable: {e}", "status": "error"}
    return render_template('index.html', data=data)

if __name__ == '__main__':
    app.run(host='0.0.0.0', port=5000)
```

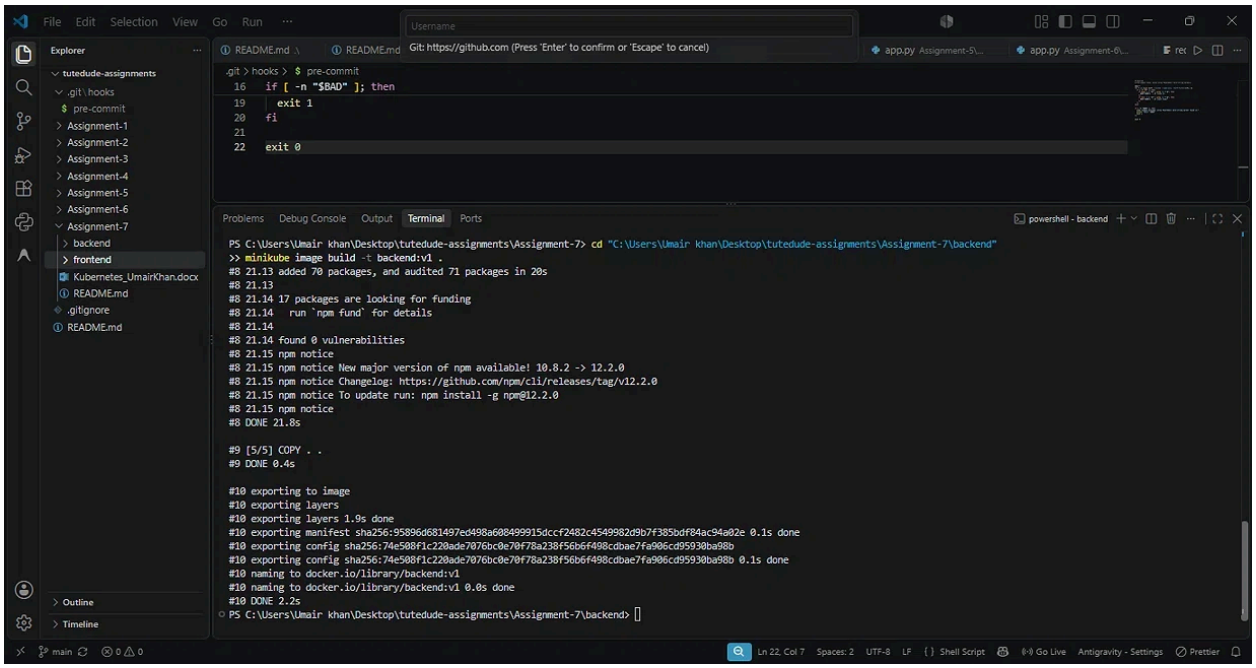
Note: The frontend reads the backend URL from the `BACKEND_URL` environment variable, which is injected via the Kubernetes Deployment manifest. The default value points to `backend-service:3000` — the Kubernetes Service DNS name.

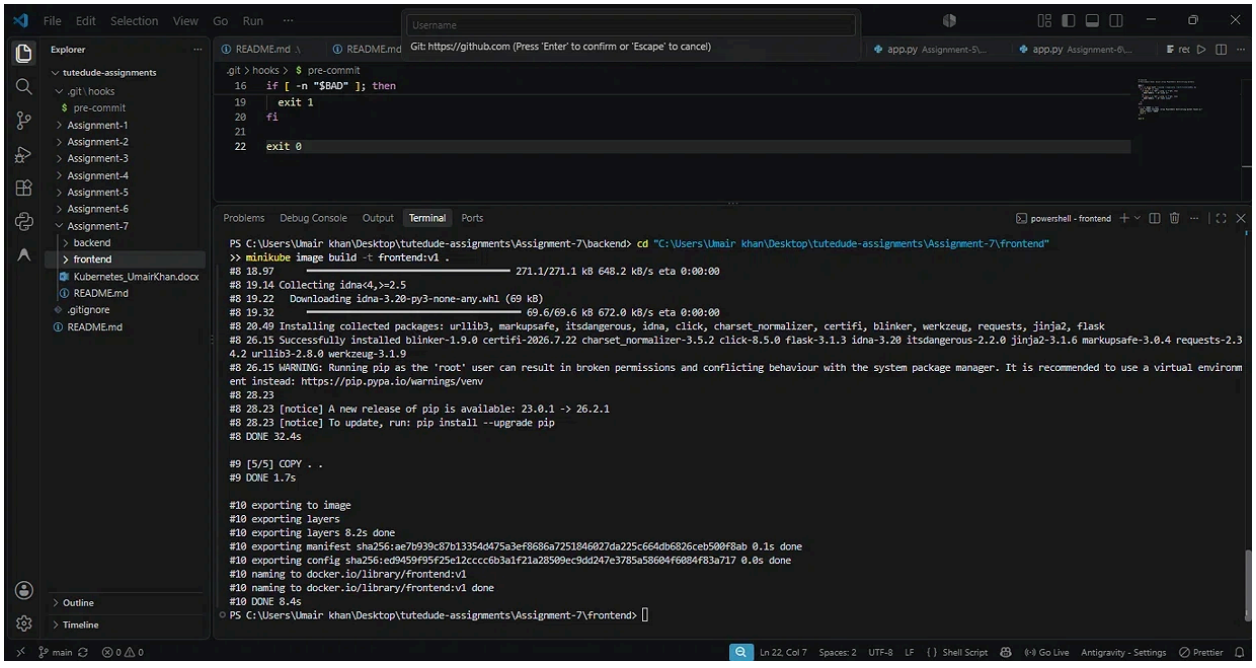
Step 4: Build Docker Images Inside Minikube

Minikube provides a built-in image builder so images are stored directly in the cluster's Docker daemon, avoiding registry pushes:

```
powershell
cd backend
minikube image build -t backend:v1 .

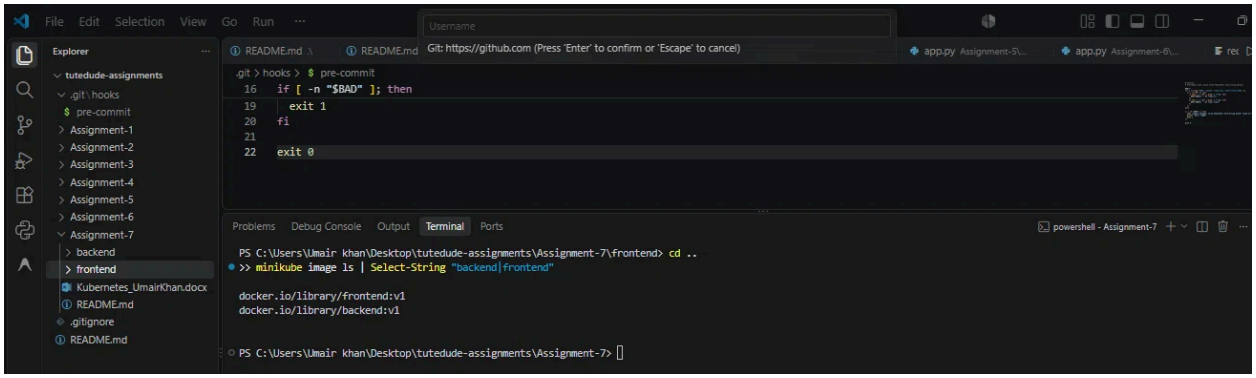
cd ..\frontend
minikube image build -t frontend:v1 .
```





Verify images exist inside Minikube:

powershell
minikube image ls | Select-String "backend|frontend"



Step 5: Write Kubernetes Manifests

Backend Deployment (`backend/deployment.yaml`):

yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deployment
  labels:
    app: backend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: backend
  template:
    metadata:
      labels:
        app: backend
    spec:
      containers:
      - name: backend
        image: backend:v1
        imagePullPolicy: Never
        ports:
        - containerPort: 3000
```

Why `imagePullPolicy: Never`? Since the image is built locally inside Minikube, Kubernetes should never attempt to pull it from a remote registry. `Never` ensures it only uses the local image.

Backend Service (`backend/service.yaml`):

```yaml
apiVersion: v1
kind: Service
metadata:
  name: backend-service
spec:
  selector:
    app: backend
  ports:
  - protocol: TCP
    port: 3000
    targetPort: 3000
  type: ClusterIP
```

ClusterIP is used because the backend is only accessed internally by the frontend pods — no external exposure needed.

Frontend Deployment (`frontend/deployment.yaml`):

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: frontend-deployment
  labels:
    app: frontend
spec:
  replicas: 2
  selector:
    matchLabels:
      app: frontend
  template:
    metadata:
      labels:
        app: frontend
    spec:
      containers:
      - name: frontend
        image: frontend:v1
        imagePullPolicy: Never
        ports:
        - containerPort: 5000
        env:
        - name: BACKEND_URL
```

value: "http://backend-service:3000"

The `env` section injects the backend service URL, which the Flask app reads from `os.getenv("BACKEND_URL")`.

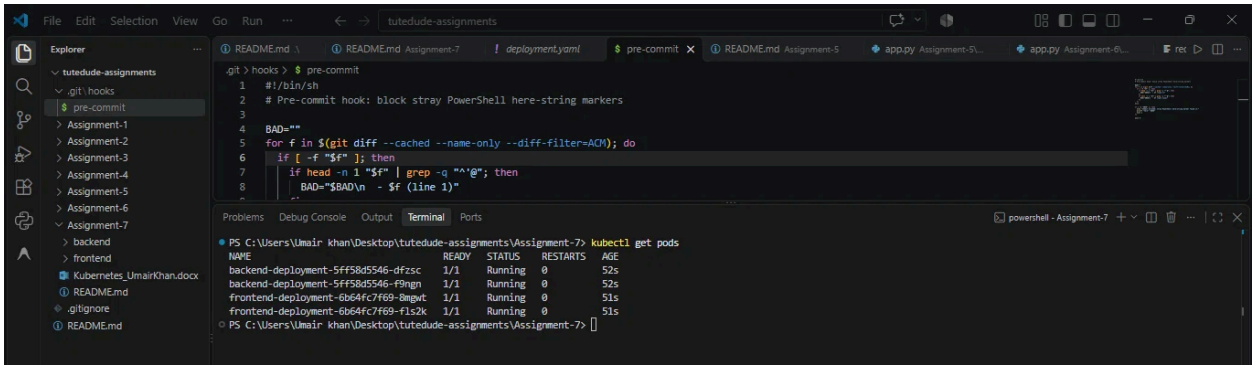
Frontend Service (`frontend/service.yaml`):

yaml
apiVersion: v1
kind: Service
metadata:
  name: frontend-service
spec:
  selector:
    app: frontend
  ports:
  - protocol: TCP
    port: 5000
    targetPort: 5000
  type: NodePort

NodePort exposes the frontend outside the cluster so it can be accessed from the host browser.

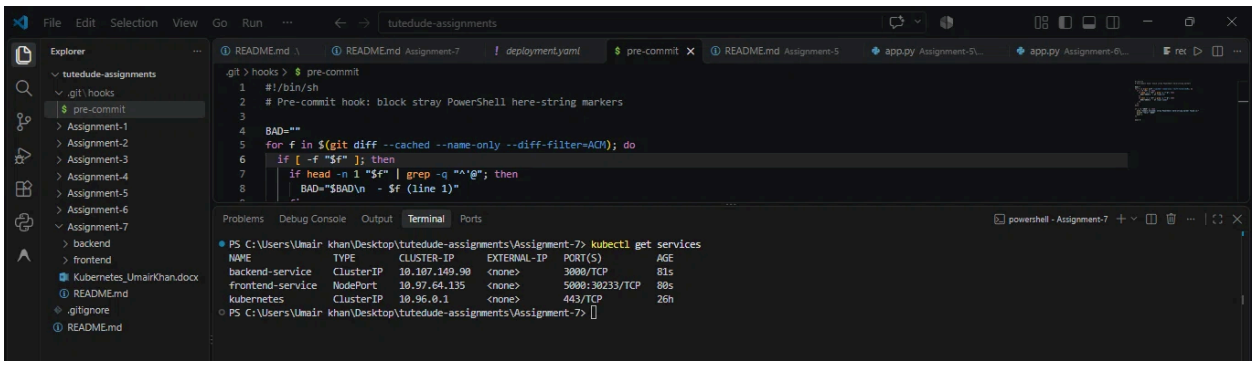
Step 6: Apply Manifests to the Cluster

powershell
kubectl apply -f backend/deployment.yaml
kubectl apply -f backend/service.yaml
kubectl apply -f frontend/deployment.yaml
kubectl apply -f frontend/service.yaml



Step 7: Verify Pods Are Running

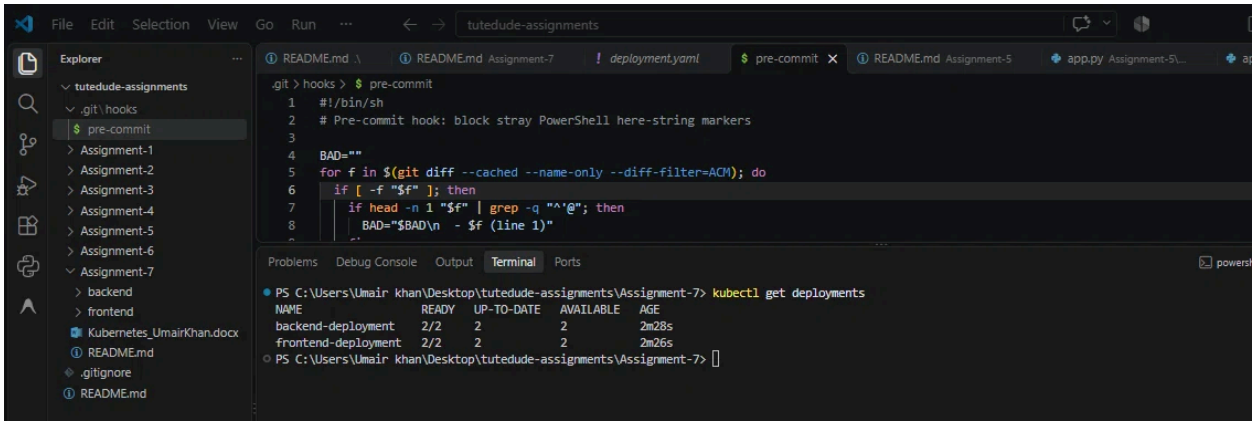
powershell
kubectl get pods

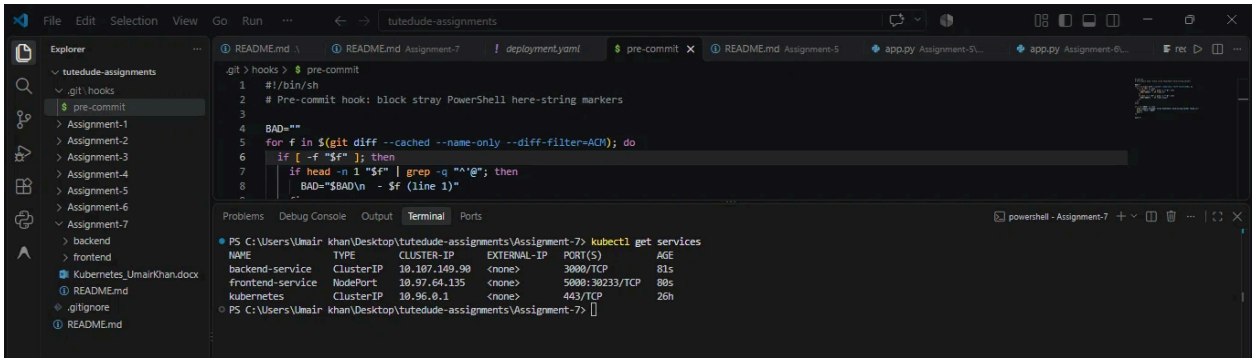


Result: **4 pods running** (2 backend replicas + 2 frontend replicas), all 1 / 1 Ready with 0 restarts.

Step 8: Verify Deployments and Services

powershell
kubectl get deployments
kubectl get services

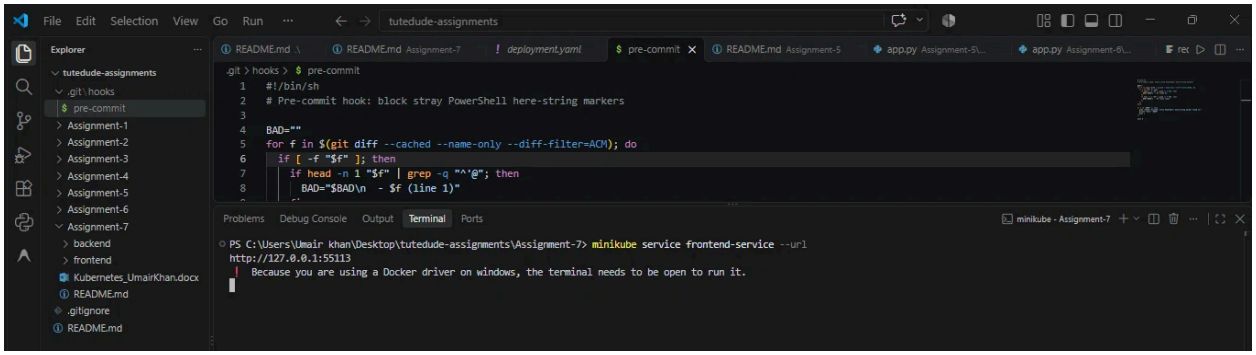




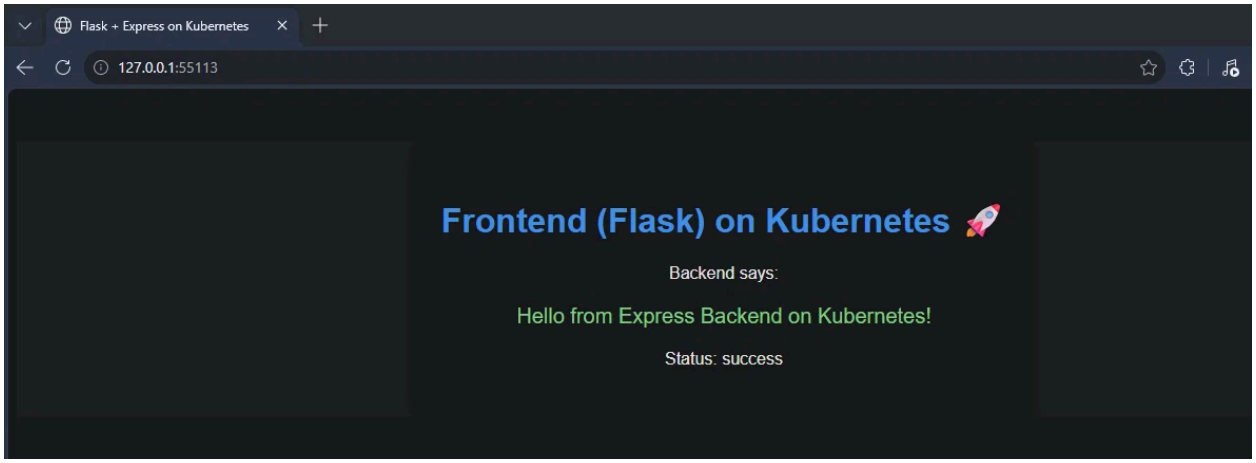
Step 9: Expose Frontend and Access the Application

powershell
minikube service frontend-service --url

This creates a tunnel from the local machine to the NodePort on the Minikube cluster and returns a URL (e.g., `http://127.0.0.1:55113`).



Open the URL in a browser:

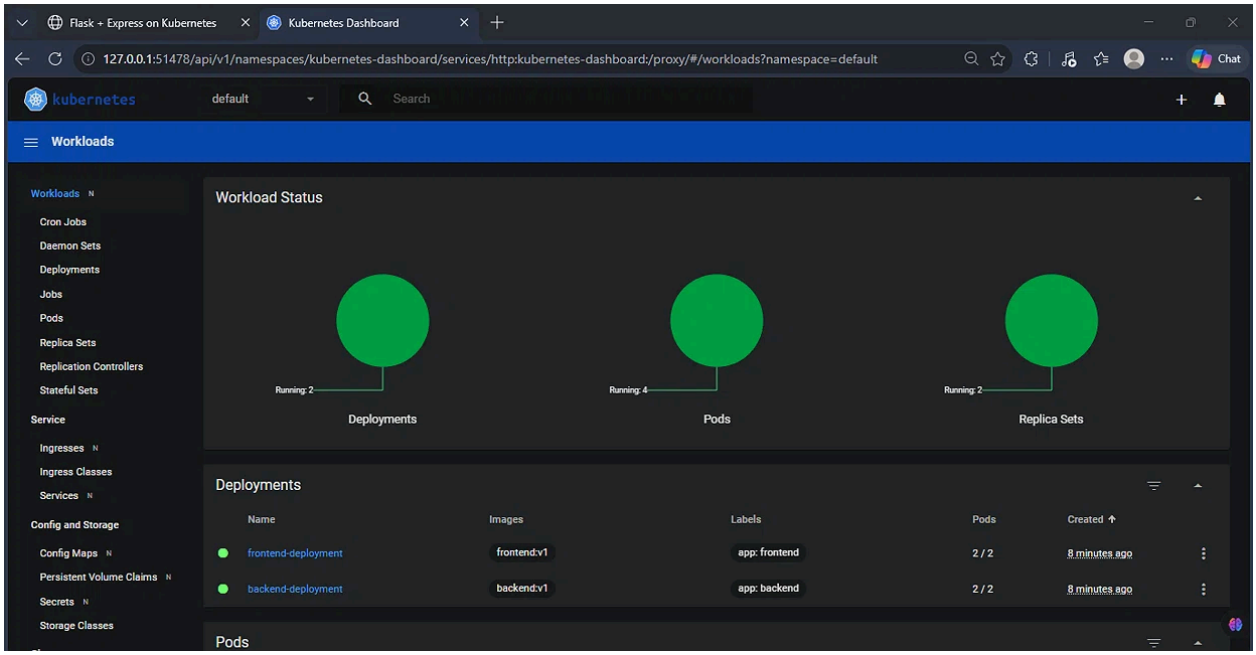


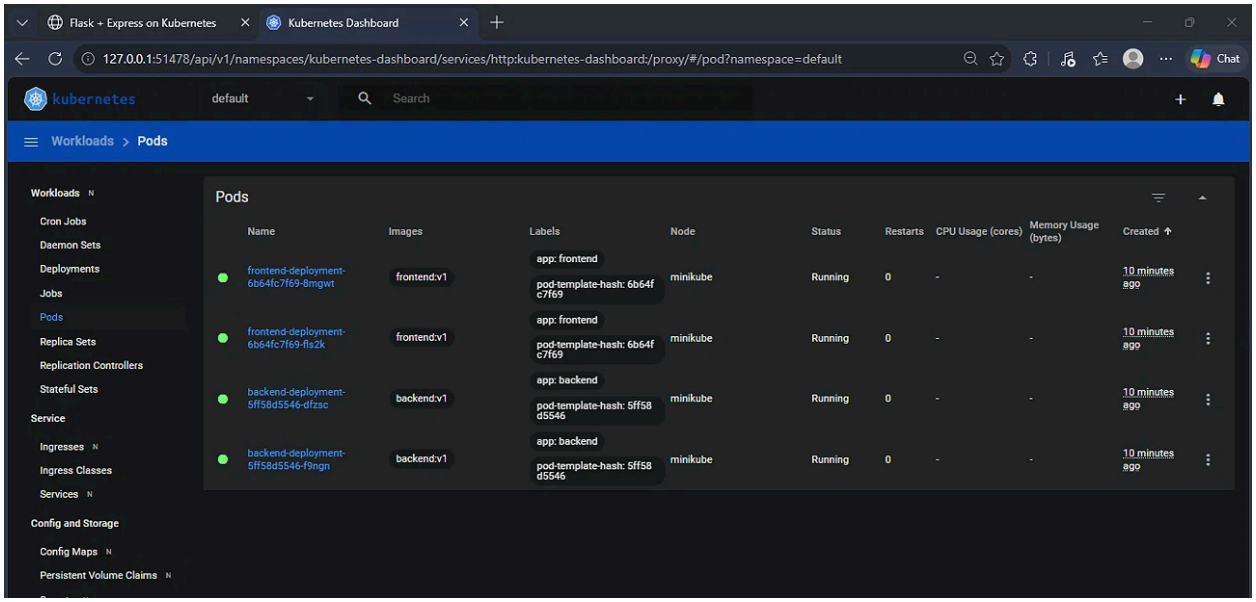
This confirms end-to-end communication between the frontend and backend pods via Kubernetes service discovery.

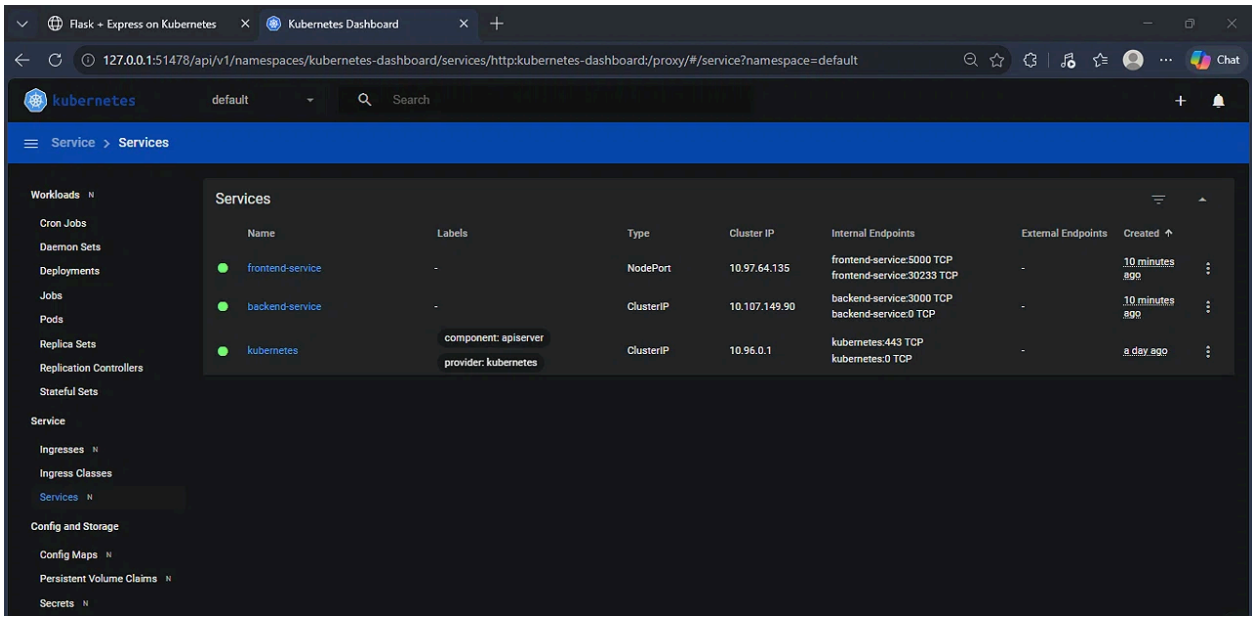
Step 10: Kubernetes Dashboard

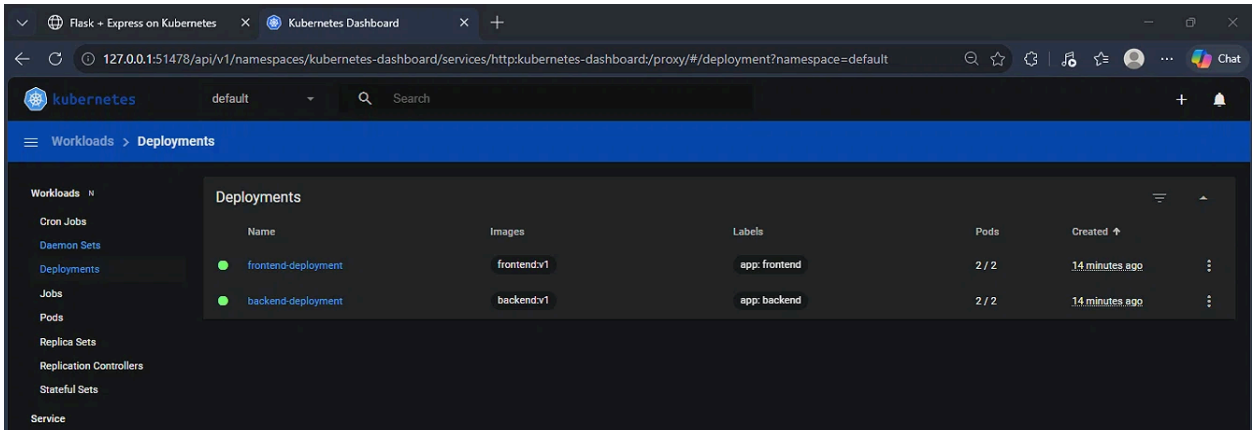
Launched the official Kubernetes Dashboard:

powershell
minikube dashboard



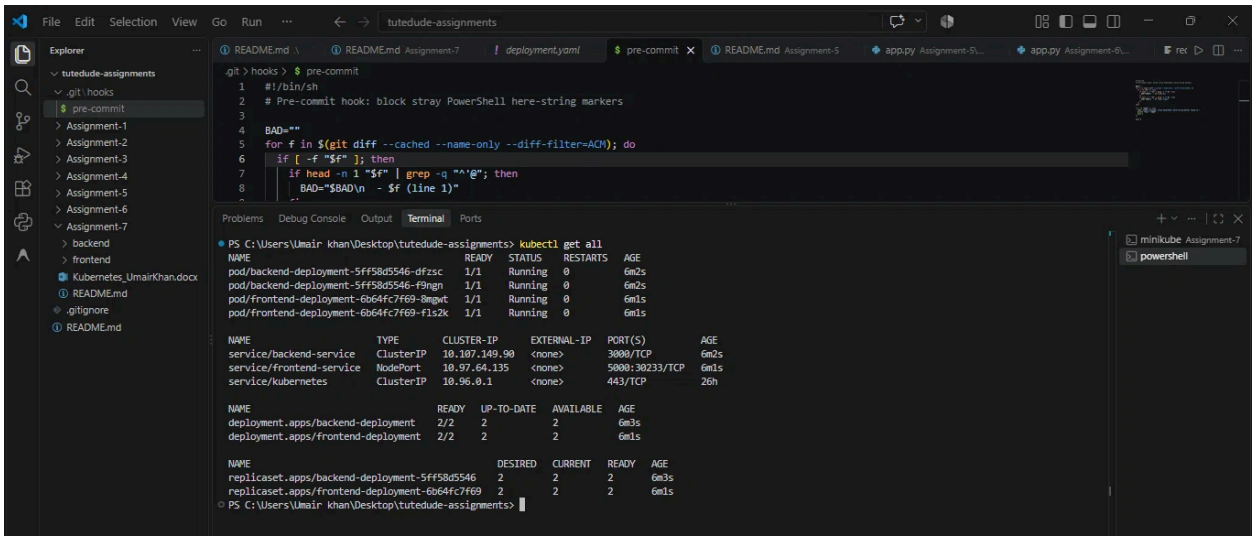

Flask + Express on Kubernetes
Kubernetes Dashboard
127.0.0.1:51478/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/#/pod?namespace=default
kubernetes
default
Search
Workloads > Pods
Workloads
Cron Jobs
Daemon Sets
Deployments
Jobs
Pods
Replica Sets
Replication Controllers
Stateful Sets
Service
Ingresses
Ingress Classes
Services
Config and Storage
Config Maps
Persistent Volume Claims
Pods
Name
Images
Labels
Node
Status
Restarts
CPU Usage (cores)
Memory Usage (bytes)
Created
frontend-deployment-6b64fc7f69-8mgwt
frontend:v1
app: frontend
pod-template-hash: 6b64fc7f69
minikube
Running
0
10 minutes ago
frontend-deployment-6b64fc7f69-fls2k
frontend:v1
app: frontend
pod-template-hash: 6b64fc7f69
minikube
Running
0
10 minutes ago
backend-deployment-5ff58d5546-dfzsc
backend:v1
app: backend
pod-template-hash: 5ff58d5546
minikube
Running
0
10 minutes ago
backend-deployment-5ff58d5546-f9ngn
backend:v1
app: backend
pod-template-hash: 5ff58d5546
minikube
Running
0
10 minutes ago


Flask + Express on Kubernetes
Kubernetes Dashboard
127.0.0.1:51478/api/v1/namespaces/kubernetes-dashboard/services/http:kubernetes-dashboard:/proxy/#/service?namespace=default
kubernetes
default
Search
Service > Services
Workloads
Cron Jobs
Daemon Sets
Deployments
Jobs
Pods
Replica Sets
Replication Controllers
Stateful Sets
Service
Ingresses
Ingress Classes
Services
Config and Storage
Config Maps
Persistent Volume Claims
Secrets
Services
Name
Labels
Type
Cluster IP
Internal Endpoints
External Endpoints
Created
frontend-service
-
NodePort
10.97.64.135
frontend-service:5000 TCP
frontend-service:30233 TCP
-
10 minutes ago
backend-service
-
ClusterIP
10.107.149.90
backend-service:3000 TCP
backend-service:0 TCP
-
10 minutes ago
kubernetes
component: apiserver
provider: kubernetes
ClusterIP
10.96.0.1
kubernetes:443 TCP
kubernetes:0 TCP
-
a day ago



Step 11: Complete Cluster State

powershell
kubectl get all



---

6. Challenges and Troubleshooting

Challenge 1: Stray PowerShell Here-String Markers in Files

When initially creating the YAML, Dockerfiles, and source files via PowerShell here-strings, several files were inadvertently written with `'@` (the here-string terminator) as the first line. This caused the Docker build to fail with:

text
dockerfile parse error on line 1: unknown instruction: '@

Resolution: A byte-level PowerShell script was written to detect and strip any line consisting solely of `'@` from the first or last line of each file. This resolved the parsing errors and the images built successfully.

Challenge 2: Minikube `docker-env` Bug on Windows

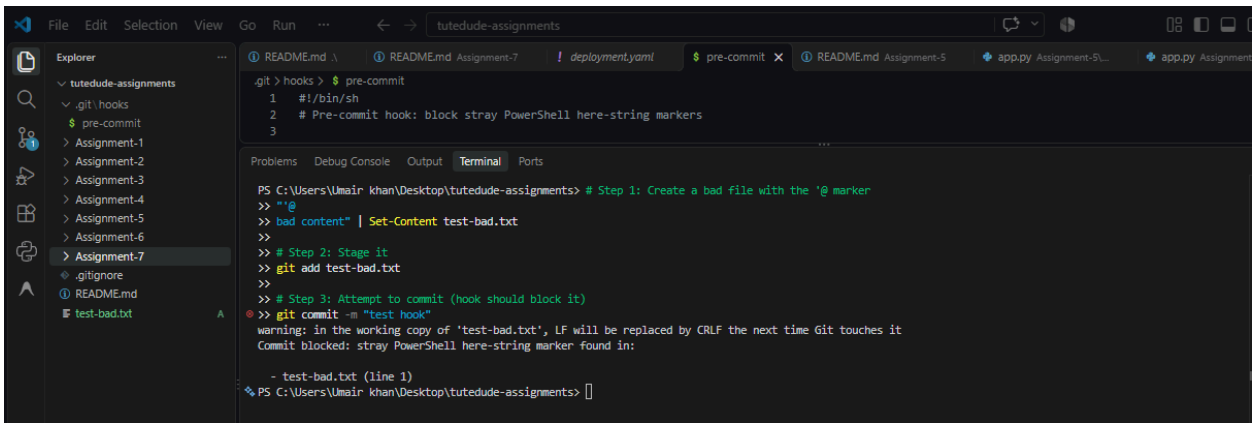
`minikube docker-env | Invoke-Expression` failed on Windows with:

text
❌ Exiting due to SSH_AGENT_START: starting an SSH agent on Windows is not yet supported

Resolution: Used the built-in `minikube image build` command instead, which builds images directly inside Minikube's Docker daemon without needing to modify the local environment.

Challenge 3: Preventing Future Occurrences — Pre-commit Hook

A **Git pre-commit hook** was installed at `.git/hooks/pre-commit` that scans all staged files for stray `'@` markers and blocks the commit if any are found. This prevents the same bug from being pushed to the repository in the future.



---

7. Key Learnings

1. **Containerization:** Successfully built Docker images for a Node.js backend and a Python/Flask frontend.

2. **Kubernetes Manifests:** Wrote YAML for Deployments (replica management) and Services (ClusterIP for internal, NodePort for external access).
3. **Service Discovery:** Kubernetes internal DNS allows pods to reach each other by service name — `http://backend-service:3000` works across pods without hardcoding IPs.
4. **Image Handling in Minikube:** Building images directly into the cluster's daemon with `minikube image build` is the recommended approach on Windows where `docker-env` has known issues.
5. **Debugging YAML/Dockerfile Errors:** Learned to inspect file bytes and fix parsing issues at the byte level.
6. **DevOps Safety Practices:** Added a pre-commit hook to catch future file corruption issues before they reach the repository.

---

8. Conclusion

This assignment demonstrated a complete end-to-end DevOps workflow for deploying a full-stack application on Kubernetes. Both the Flask frontend and Express backend were containerized, deployed as Kubernetes Deployments with 2 replicas each, and exposed via Services. The application successfully communicates across pods using Kubernetes DNS service discovery. The final state shows a healthy cluster with 4 running pods, 2 services, and 2 deployments — all accessible via the Minikube NodePort tunnel.

---

9. GitHub Repository

Full source code, Dockerfiles, and Kubernetes manifests are available at:

https://github.com/Umair-bey/tutedude-assignments/tree/main/Assignment-7

---

10. References

- Kubernetes Documentation: https://kubernetes.io/docs/
- Minikube Documentation: https://minikube.sigs.k8s.io/docs/
- Docker Documentation: https://docs.docker.com/

✅ END OF DOCUMENT

🎯 What to Do Now

1. **Open Microsoft Word** (or Google Docs)
2. **Save as** `Kubernetes_UmairKhan.docx` inside `C:\Users\Umair khan\Desktop\tutedude-assignments\Assignment-7\`
3. **Copy each section** from above into the doc
4. **Paste screenshots** at every `[INSERT SCREENSHOT: ...]` marker
5. **Bold headings** as needed (or use Word's heading styles)
6. **Save**

🎯 Then — Create the Zip for Submission

powershell
cd "C:\Users\Umair khan\Desktop\tutedude-assignments"
Compress-Archive -Path Assignment-7 -DestinationPath Kubernetes_UmairKhan.zip -Force

Then verify:

powershell
Get-ChildItem Kubernetes_UmairKhan.zip

Reply when the doc is done, and I'll give you the final checklist + submission steps.

Great work on the entire deployment. 🎉